MANIPAL UNIVERSITY JAIPUR

University u der Sectio 2(f) of the UGC Act

NAME	HIRDYANSH VARSHNEY
ROLL NO.	2414500172
PROGRAMME	BCA
SEMESTER	4th
SUBJECT NAME	System Software
SUBJECT CODE	DCA2206

SET – 1

Q1. Functional Units of 8086 Microprocessor: Role of BIU and EU

- **Overview**

The 8086 is Intel's 16-bit microprocessor released in 1978. What makes it architecturally special is the way it divides its internal operations into two separate functional units that can work simultaneously. Instead of fetching one instruction and executing it before moving to the next, the 8086 can do both at the same time, which speeds up overall processing. These two units are the Bus Interface Unit (BIU) and the Execution Unit (EU).

- **Bus Interface Unit (BIU)**

The BIU acts as the communication layer between the 8086 processor and its external environment, meaning memory and I/O devices. It handles all the activities related to reading instructions from memory and transferring data. Inside the BIU, there is a 6-byte instruction queue that stores prefetched instructions. The BIU continuously fills this queue whenever it is not full, even while the EU is busy executing previous instructions.

The BIU contains four segment registers: CS (Code Segment), DS (Data Segment), SS (Stack Segment), and ES (Extra Segment). These registers define the base addresses of different memory segments. Along with these, the BIU also contains the Instruction Pointer (IP), which

always points to the memory address of the next instruction to be fetched. The address for fetching is formed by combining the CS register and the IP value.

An important point is that the BIU and EU do not wait for each other. If the instruction queue is full, the BIU simply pauses until space becomes available. If the EU empties the queue faster than the BIU fills it, the BIU increases its fetching. This dynamic coordination is what makes the 8086 efficient.

- **Execution Unit (EU)**

The EU takes instructions from the instruction queue provided by the BIU and actually executes them. It never directly accesses memory or I/O ports. When data from memory is required, the EU sends a request to the BIU, which then performs the actual memory read or write. This separation of duties keeps the EU focused only on processing.

Inside the EU, the main component is the ALU (Arithmetic and Logic Unit), which does all calculations and logical comparisons. The EU also has general purpose registers: AX (Accumulator), BX (Base), CX (Counter), and DX (Data). These 16-bit registers can also be split into two 8-bit halves for byte-level operations. Additionally, the EU has pointer registers like SP (Stack Pointer), BP (Base Pointer), and index registers SI and DI. The Flags Register inside the EU reflects the outcome of operations, such as whether a result was zero, negative, or caused an overflow.

- **Pipelining Through BIU-EU Coordination**

The real benefit of this two-unit design is a form of pipelining. Suppose the EU is executing instruction 1. At the same time, the BIU is already fetching instructions 2, 3, and maybe 4 into the queue. When the EU finishes instruction 1, it immediately picks up instruction 2 from the queue without any delay. This overlap makes the 8086 significantly faster than a processor that would have to stop execution just to go fetch the next instruction from memory.

Conclusion

The 8086 microprocessor's internal division into BIU and EU is a well-thought-out design that uses instruction prefetching to reduce idle time. The BIU manages all memory and I/O communication while the EU handles pure computation. Together they implement a basic pipeline that improves the overall throughput of the processor.

Q2. Forward Reference Problem in Single Pass Assembler and Handling of Undefined Symbols

- **What is a Single Pass Assembler?**

A single pass assembler reads the source assembly program from top to bottom exactly one time and generates the object code during that single scan. This makes it fast and memory efficient. However, it runs into a fundamental difficulty when a symbol or label is used before it appears in the code. This difficulty is known as the forward reference problem.

- **Understanding the Forward Reference Problem**

Consider a situation where a program has an instruction like BRANCH DONE near the beginning of the code, and the label DONE is defined at a line that appears much later. When the assembler reaches the BRANCH instruction during its single pass, it needs to put the address of DONE in the machine instruction. But since DONE has not been seen yet, its address is unknown. The assembler has no way of knowing where DONE will be.

This is the core of the forward reference problem. In a two-pass assembler, pass one creates a complete symbol table with all label addresses, so by pass two every address is known. In single pass, there is no such luxury. The assembler must generate machine code on the fly and handle unknowns intelligently.

- **How Undefined Symbols are Handled**

The standard method used by single pass assemblers is a technique called backpatching. When the assembler encounters a forward reference, it does two things: first, it generates the machine instruction but puts a placeholder value (usually zero or a blank) in the address field. Second, it creates an entry in a data structure called the Incomplete Instruction Table or Forward Reference Table. This entry stores the memory location of the incomplete instruction and the name of the symbol that needs to be resolved.

The symbol table in a single pass assembler is also designed to store such unresolved symbols with a list of locations that are waiting for their address. When the assembler eventually reaches the definition of the symbol (i.e., the label), it immediately updates all the instructions that were waiting for it. The actual addresses are filled in retrospectively. This filling-in process is called backpatching.

For example, when DONE is finally encountered in the code and its address is determined to be, say, 250, the assembler goes back to every location stored in the forward reference list for DONE and writes 250 in those instruction fields. The object code is then complete.

- **Limitations**

Backpatching works well for symbols defined within the same module. But if a symbol is defined in a completely different file or module (an external symbol), the single pass assembler

cannot resolve it at all. In such cases, the assembler leaves a note in the object file for the linker to handle it later during the linking phase.

Conclusion

The forward reference problem is a natural challenge when processing code in a single scan. Backpatching is the practical solution that allows single pass assemblers to handle undefined symbols efficiently within the same module. Though not as complete as a two-pass approach, this technique makes single pass assembly workable for a wide range of programs.

Q3. MNT vs MDT (5 Marks) | Bootstrap Loader Working (5 Marks)

- **Part A: Macro Name Table (MNT) vs Macro Definition Table (MDT)**

Macros are a way of writing a set of instructions once and reusing them multiple times by just calling the macro name. The macro processor needs to store and manage macro information, and it uses two main tables for this: the Macro Name Table (MNT) and the Macro Definition Table (MDT).

The Macro Name Table serves as a directory of all macros in the program. For each macro, the MNT stores the macro name, the number of parameters it takes, and most importantly, a pointer that shows where the macro's actual instructions start in the MDT. Think of MNT as the index of a book. It tells you what macros exist and where to find them, but it does not contain the macro instructions itself.

The Macro Definition Table stores the actual content or body of each macro. It contains the sequence of assembly instructions that form the macro. When a macro is called anywhere in the program, the processor looks up the MNT to get the MDT pointer, then goes to that position in MDT and reads the macro body from there. The actual expansion of the macro happens using this content.

To summarize the key differences: MNT has one entry per macro and stores only names and pointers; MDT holds the full instruction sequence for every macro and is therefore much larger. MNT is searched first during a macro call; MDT is accessed second for the actual expansion. Both tables together form the core data structure of the macro processing phase.

- **Part B: Working of a Simple Bootstrap Loader**

Every time we switch on a computer, something must start the process of loading the operating system. This starting mechanism is called the bootstrap loader. The name comes from an old

expression about pulling oneself up without help, which is exactly what the computer does. It uses a small built-in program to load a larger program.

The bootstrap loader is stored in a non-volatile memory chip on the motherboard, which is the ROM or BIOS/UEFI chip. As soon as the processor receives power, it automatically jumps to a predefined memory address where the bootstrap loader's first instruction is stored. From this point, the bootstrap loader takes control.

The first task the bootstrap loader does is a hardware check called the Power On Self Test (POST). It verifies that essential components like RAM, keyboard, and storage devices are functioning properly. If any critical component fails, an error is shown. If everything is fine, the loader moves to the next stage.

Next, the bootstrap loader searches for a bootable storage device based on the boot order set in BIOS. Once it finds a bootable disk, it reads the very first sector of that disk, which is 512 bytes in size and is called the Master Boot Record (MBR). This MBR is loaded into a fixed location in RAM and then the processor transfers control to the code inside the MBR.

The MBR code is a second-stage loader that knows the file system structure of the disk. It reads the operating system kernel files from disk, loads them into RAM address by address, and handles the necessary format conversion from stored object code to executable binary. Once all essential parts of the OS are in memory, the bootstrap loader transfers control to the OS kernel's starting address. From here, the OS takes over, initializes its own structures, loads drivers, and presents the user interface. The bootstrap loader's job is then complete.

SET – 2

Q1. Program Relocation in a Linker: Translated Origin, Linked Origin, and Relocation Factor

- **Introduction**

When programmers write assembly or compiled code, the assembler generates addresses starting from a base address, typically zero, without knowing where the program will actually be placed in memory at runtime. Multiple programs share the same physical memory, so the operating system places each program at whatever free memory location is available. This means addresses in the original object code may not match the actual memory addresses used at runtime. Fixing this mismatch is called program relocation and it is one of the important jobs of the linker.

- **Translated Origin**

The translated origin is the base address that was assumed by the assembler when it created the object code. In most cases, this is address 0. So when the assembler writes an instruction like LOAD 500, it means the data is at address 500 from the assumed start of the program. Everything in the object code, including jump addresses, data addresses, and call addresses, is calculated relative to this assumed start. The translated origin is essentially the address world in which the object code was generated.

- **Linked Origin**

Real programs are rarely made of just one file. Multiple object modules are compiled separately and then linked together into one executable. During linking, each module is assigned a specific position in the final combined program. The address at which a particular module begins in the final linked output is called its linked origin. For instance, if the linker places Module A first and it is 300 bytes long, then Module B's linked origin would be 300, and if Module B is 200 bytes, Module C's linked origin would be 500, and so on.

- **Relocation Factor and the Relocation Process**

The relocation factor is simply the difference between the linked origin and the translated origin. If a module has a translated origin of 0 and its linked origin is 1500, then its relocation factor is 1500. This factor represents how much every address in the module must be shifted to reflect the actual memory position.

However, not every value in the object code represents an address. Constants, register numbers, and immediate operands should not be modified. Only those fields that contain addresses referring to locations within the program need adjustment. The assembler helps with this by generating a Relocation Table or Relocation Dictionary alongside the object code. This table marks exactly which bytes in the object code are relocatable addresses.

The linker reads this relocation table and, for each marked location, adds the relocation factor to the value stored there. For example, if an instruction refers to address 200 and the relocation factor is 1500, the linker changes it to 1700. After processing all modules this way, the final executable has correct absolute addresses that match the actual memory layout, and the program can run without address errors.

- **Conclusion**

Program relocation solves a very practical problem in software development. It allows code to be written and compiled without knowing the final memory address, and still run correctly

wherever it is placed. The translated origin, linked origin, and relocation factor together define the mathematical framework for this address adjustment process.

Q2. PCI Device Auto-Configuration at Boot and Role of Device Drivers in PCI Initialization

- **Introduction to PCI and Auto-Configuration**

PCI (Peripheral Component Interconnect) is a hardware bus standard used to connect devices like graphics cards, sound cards, and network adapters to a computer's motherboard. One of the best features of PCI is that it does not require users to manually configure device addresses or interrupt numbers. This is handled automatically during the system boot process through a mechanism known as PCI enumeration or auto-configuration.

- **PCI Configuration Space**

Every PCI device has a dedicated 256-byte area called the PCI Configuration Space. This space is separate from the device's regular operational memory and is used purely for device identification and configuration. Key fields in the configuration space include the Vendor ID, Device ID, Class Code, Subsystem ID, Base Address Registers (BARs), and the Interrupt Line field. The Vendor ID uniquely identifies the manufacturer, and the Device ID identifies the specific product. These two together help the OS find the correct driver.

- **Auto-Configuration Process During Boot**

During system startup, either the BIOS or the OS performs a structured scan of all PCI buses. PCI supports up to 256 buses, each with up to 32 devices, and each device can have up to 8 functions. The configuration software reads through all possible bus, device, and function number combinations. For each combination, it tries to read the Vendor ID from the configuration space. A value of 0xFFFF means no device is present at that slot. Any other value indicates a real device.

Once a device is detected, the system reads its Class Code to understand what kind of device it is. Then it programs the BARs to assign memory address ranges and I/O port ranges to the device. This tells the device where in the system address space it should listen for commands. The system also assigns an interrupt line to each device so the device can use hardware interrupts to communicate with the CPU. By the time the OS starts loading, all this configuration is already done by the firmware and the devices are ready to be used.

- **Role of Device Drivers in PCI Initialization**

Even though the hardware is configured, the operating system cannot use a PCI device without a corresponding device driver. The OS maintains a database of drivers matched to Vendor ID and Device ID combinations. When the OS discovers a PCI device, it looks up this database to find the right driver. If the driver is found, it is loaded into memory.

The driver then reads the device's configuration space to learn its assigned memory addresses, I/O ports, and interrupt line. It maps these resources into the kernel's address space and registers interrupt handlers. This gives the OS a way to send commands to the device and receive responses. The driver presents a standard interface to the rest of the OS so that other software does not need to know the low-level details of the hardware.

- **Conclusion**

PCI auto-configuration is a powerful feature that significantly simplifies hardware management. The combination of structured configuration space, BIOS-level enumeration, and OS device driver loading creates a seamless experience where new hardware is detected and made usable automatically. Device drivers are the final and critical piece that make this hardware accessible to software applications.

Q3. Android OS Security (5 Marks) | SSDP vs UDDI Discovery (5 Marks)

- **Part A: Security in Android Operating System**

Android runs on more smartphones than any other operating system in the world. This enormous reach makes it a prime target for attackers, which is why Android has been designed with multiple layers of security to protect both user data and device integrity.

At the foundation, Android is built on the Linux kernel, which provides process isolation, user-level permissions, and memory protection. Each application installed on an Android device is treated like a separate user in the Linux sense. It gets its own unique user ID and runs in its own sandboxed process. This means no app can access the files or memory of another app unless the system explicitly allows it through defined channels. This Application Sandbox is the most fundamental security boundary in Android.

Android also has a strict Permission Model. Apps must declare in their manifest file what system resources they plan to use. Users are asked to grant these permissions either during installation or at runtime when the app actually tries to use the resource. For example, when a messaging app first tries to access your contacts, a dialog box appears asking the user to approve or deny. This prevents apps from silently accessing sensitive data.

Threats that Android faces include Trojan apps that look genuine but carry malware, spyware that monitors user activity, ransomware that locks user data, and phishing pages designed to steal credentials. Android responds with Google Play Protect, which scans all installed and newly downloaded apps for malicious behavior. SafetyNet and Play Integrity APIs help apps verify whether the device is genuine and unmodified. Verified Boot ensures that the OS itself has not been tampered with between reboots. Additionally, modern Android versions use file-based encryption so that data is protected even if someone physically accesses the storage.

Overall, Android's security approach is layered, meaning even if one defense is bypassed, others remain active. This defense-in-depth strategy is what makes Android reasonably secure for daily use despite the large number of threats targeting it.

- **Part B: SSDP (Local Discovery) vs UDDI (Internet-Wide Discovery)**

When services need to be found on a network, discovery protocols are used. Depending on whether the discovery is within a local network or across the internet, different protocols are used. SSDP and UDDI represent these two very different approaches.

SSDP stands for Simple Service Discovery Protocol and it operates within a local area network. It is designed for scenarios like finding a printer in an office, connecting to a smart TV, or discovering IoT devices on a home Wi-Fi network. SSDP works using IP multicast. A device that wants to announce its services broadcasts an ssdp:alive notification to the multicast address. A client looking for services can send an M-SEARCH multicast request, and any matching device responds with details about itself. Because it uses multicast, SSDP does not need a central server. It is decentralized and works purely within the network boundary. SSDP is the core protocol used in UPnP (Universal Plug and Play) which is common in home networks and consumer electronics.

UDDI stands for Universal Description, Discovery, and Integration. It was designed for a completely different scale and use case, specifically discovering web services over the internet in business environments. UDDI uses a centralized registry model. A company that offers a web service registers it in a UDDI registry with a detailed description of what the service does, how to call it, and where it is hosted. Another company looking for such a service searches the UDDI registry, finds a match, downloads the service description (usually in WSDL format), and uses it to integrate the service into their own system. UDDI was part of the early web services ecosystem alongside SOAP and WSDL, but it has become less popular today as developers prefer simpler REST APIs for service integration.

The core distinction is: SSDP is real-time, local, multicast-based, and serverless, suited for device discovery in small networks. UDDI is registry-based, global, structured, and designed for long-term business service cataloging across the internet. Both serve the purpose of discovery but for completely different environments and scales.